1. Introduction to C

C is a general-purpose, procedural programming language. It was developed for writing system software such as operating systems and compilers, and it is still used wherever speed and direct control over memory matter.
C is called a middle-level language. It gives the readable structure of a high-level language, and at the same time allows low-level operations such as direct memory access through pointers.

History of C

Year	Language / Event	Developed by
1966	BCPL	Martin Richards, Cambridge
1969	B language	Ken Thompson, Bell Laboratories
1972	C language created	Dennis Ritchie, Bell Laboratories
1973	UNIX operating system rewritten in C	Thompson and Ritchie
1978	"The C Programming Language" (K&R C)	Kernighan and Ritchie
1989	ANSI C standard (C89)	ANSI committee X3J11
1990	ISO C standard (C90)	ISO
1999	C99 standard	ISO
2011	C11 standard	ISO
2018	C17 standard	ISO

C was derived from B, which was derived from BCPL. B could not handle different types of data because it was typeless, so Ritchie added data types and other features, and named the result C.

Features of C

- Simple and reliable — a small set of keywords and a clear structure.
- Portable — a C program written on one machine can be compiled and run on another with little or no change.
- Fast and efficient — compiled directly to machine code, so execution is quick.
- Structured — a program is divided into functions and blocks, which makes it easier to write and debug.
- Rich set of operators — about 45 operators, including bitwise operators.
- Rich library — a large collection of built-in functions such as printf(), scanf() and sqrt().
- Memory management — pointers and dynamic allocation give direct control over memory.
- Extensible — a programmer can add their own functions to the library.

- Middle-level — supports both high-level structure and low-level operations.

2. Structure of a C Program

Every C program follows a fixed arrangement of sections. Only the main() function is compulsory; the other sections are used when the program needs them.

Section	Purpose	Compulsory
Documentation	Comments giving the program name, author and purpose	No
`Preprocessor / Link`	Header files included using #include	Usually yes
Definition	Symbolic constants defined using #define	No
Global declaration	Variables and function declarations used by all functions	No
`main() function`	Execution of every C program begins here	Yes
`Sub-program`	User-defined functions written after main()	No

Example

```
/* area.c - structure of a C program
   Author: BCA I Sem   */

#include <stdio.h>          /* Preprocessor section */
#define PI 3.14159          /* Definition section   */

float radius = 2.5;         /* Global declaration   */

int main()                  /* main() function      */
{
    float area;             /* Local declaration    */
    area = PI * radius * radius;
    printf("Area of circle = %f", area);
    return 0;
}
```

Output

```
Area of circle = 19.634937
```

Explanation

- /* ... */ is a comment. The compiler ignores it.
- #include <stdio.h> tells the preprocessor to include the standard input/output header, which contains printf() and scanf().
- #define PI 3.14159 creates a symbolic constant. Every PI in the program is replaced by 3.14159 before compilation.
- radius is a global variable, so it is available to every function in the file.
- int main() is where execution begins. int means main() returns an integer.
- The statements inside { } form the body of main(). Each statement ends with a semicolon.
- return 0; sends 0 back to the operating system, meaning the program ended

normally.

3. Creating and Executing a C Program

A C program passes through four steps before it produces a result.

1. **Creating** - type the program in a text editor or IDE and save it with a **.c** extension, for example hello.c. This file is called the source code.
2. **Preprocessing -** preprocessor handles directives starting with #, such as #include and #define. It expands header files, replaces macros, and removes comments, producing an expanded source **.i file**
3. **Compiling** - the compiler translates the source code into machine code. If there is a syntax error, the compiler reports it and no output file is produced. This step checks for syntax errors and generates a **.s** file containing human-readable assembly instructions
4. **Linking** - the linker joins the compiled code with the library functions used by the program, such as printf(), and produces an executable file.
5. **Executing** - the executable file is loaded into memory and run. The result appears on the screen. (e.g., **.exe** on Windows or **.out** on Linux)

Commands used

Step	Command (GCC)	Result
Compile and link	gcc hello.c -o hello	creates the executable hello
`Run (Linux / macOS)`	./hello	displays the output
`Run (Windows)`	hello.exe	displays the output

Example

```
/* hello.c - the first C program */
#include <stdio.h>

int main()
{
    printf("Hello, World!");
    return 0;
}
Output
Hello, World!
```

4. Compilation Process in C

Compilation is not a single step. The source file passes through four stages, and each stage produces a different file.

Stage	Performed by	Input	Output
Preprocessing	Preprocessor	hello.c	hello.i (expanded source)
Compilation	Compiler	hello.i	hello.s (assembly code)
Assembly	Assembler	hello.s	hello.o (object code)
Linking	Linker	hello.o + libraries	hello (executable)

What happens at each stage

- Preprocessing — header files are inserted, macros defined by #define are replaced by their values, and comments are removed.
- Compilation — the expanded source is checked for syntax and converted into assembly language for the target processor.
- Assembly — the assembler converts assembly code into machine instructions and stores them as object code.
- Linking — the linker joins the object code with library code. This is why printf() works even though it is never defined in the program.

Seeing the stages

```
gcc -E hello.c -o hello.i      /* stop after preprocessing */
gcc -S hello.i -o hello.s      /* stop after compilation    */
gcc -c hello.s -o hello.o      /* stop after assembly       */
gcc hello.o -o hello           /* link and produce the exe  */
```

Execution of the program

Compilation only produces the executable file. The program actually runs when the operating system loads and executes that file. Execution passes through six stages.

Stage	What happens
Loading	The loader of the operating system copies the executable file from disk into main memory (RAM).
Process creation	The operating system creates a process for the program and allocates memory for its code, data, stack and heap.
Program initialization	The runtime environment sets up the libraries and then calls main().
Execution	The CPU fetches, decodes and executes the machine instructions one by one.
Output	The program produces its result on the screen, or writes it to a file.
Termination	After main() finishes and returns 0, the process ends and the operating system releases the memory it had allocated.

Compilation and execution at a glance

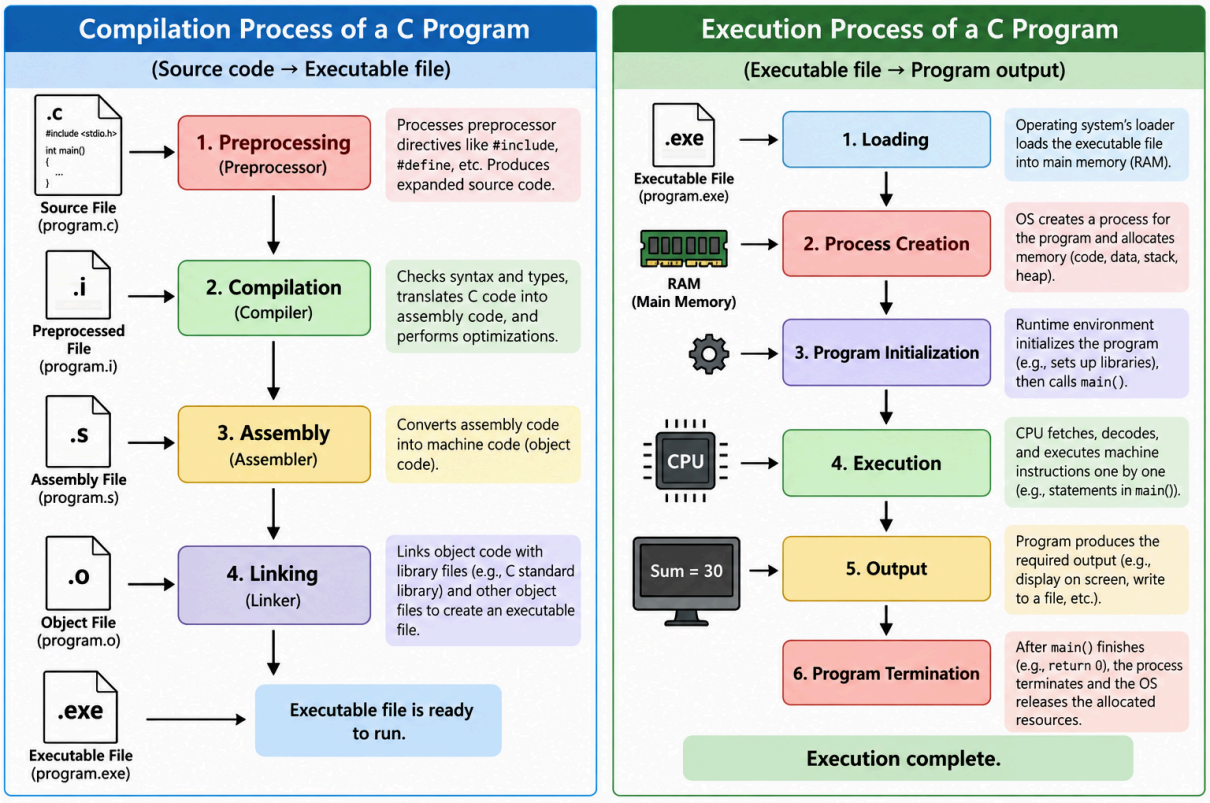


Figure 1: Compilation converts the source file into an executable file; execution turns that executable file into program output.

5. C Character Set

The character set is the group of characters the compiler recognises. Anything outside this set cannot be used in a C program, except inside comments and strings.

Type	Characters
Letters	A to Z and a to z
Digits	0 to 9
Special characters	, . ; : ? ' " ! | / \ ~ _ $ % # & ^ * - + < > ( ) [ ] { }
White space	blank space, horizontal tab, new line, carriage return, form feed

C is case sensitive. The names total, Total and TOTAL refer to three different variables.

6. C Tokens

A token is the smallest individual unit of a C program. The compiler breaks the source code into tokens before translating it. C has six types of tokens.

Token	Meaning	Example
Keyword	Reserved word with a fixed meaning	int, for, return
Identifier	Name given by the programmer	sum, total_marks
Constant	Fixed value that does not change	25, 3.14, 'A'
String	Sequence of characters in double quotes	"Hello"
Operator	Symbol that performs an operation	+, -, *, &&
Special symbol	Symbol with a special role	{ } [ ] ( ) , ;

Keywords

Keywords are reserved words whose meaning is already known to the compiler. They must be written in lower case and cannot be used as variable names. Standard C (C89) has 32 keywords.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Identifiers

An identifier is the name given to a variable, function, array or any other user-defined item. The following rules must be followed.

- The first character must be a letter or an underscore. It cannot be a digit.
- After the first character, letters, digits and underscores are allowed.
- No special character or blank space is allowed, except the underscore.
- A keyword cannot be used as an identifier.
- C is case sensitive, so marks and Marks are different identifiers.

Valid	Invalid	Reason
total	2total	begins with a digit
_count	total marks	contains a blank space
studentAge	total-marks	contains a special character

roll_no1	float	it is a keyword

Constants

A constant is a value that does not change during execution.

- Integer constant — a whole number without a decimal point, such as 25, -7, 0.
- Real (floating point) constant — a number with a decimal point or in exponential form, such as 3.14, -0.5, 1.5e3.
- Character constant — a single character in single quotes, such as 'A', '7', '$'.
- String constant — a sequence of characters in double quotes, such as "BCA".

A character constant is stored as its ASCII code, so it can be used in arithmetic.

Example

```
#include <stdio.h>

int main()
{
    char ch = 'A';
    int  code;

    code = ch;              /* char is stored as its ASCII value */

    printf("Character : %c\n", ch);
    printf("ASCII code: %d\n", code);
    printf("Next char : %c\n", ch + 1);
    printf("'a' code  : %d\n", 'a');
    printf("'0' code  : %d\n", '0');
    return 0;
}
```

Output

```
Character : A
ASCII code: 65
Next char : B
'a' code  : 97
'0' code  : 48
```

7. Data Types

A data type tells the compiler what kind of value a variable will hold and how much memory to reserve for it. C data types are grouped into four categories.

Category	Types
Primary (basic)	int, float, double, char
Derived	array, pointer, function
User-defined	structure, union, enum, typedef
Empty (void)	void

Primary data types

Data type	Size	Range	Format specifier
char	1 byte	−128 to 127	%c
unsigned char	1 byte	0 to 255	%c
short int	2 bytes	−32,768 to 32,767	%hd
int	4 bytes	−2,147,483,648 to 2,147,483,647	%d
unsigned int	4 bytes	0 to 4,294,967,295	%u
long int	4 or 8 bytes	depends on the machine	%ld
float	4 bytes	3.4E−38 to 3.4E+38 (6 digits)	%f
double	8 bytes	1.7E−308 to 1.7E+308 (15 digits)	%lf
long double	10 or 16 bytes	greater than double	%Lf

Sizes are machine dependent. The sizeof operator returns the exact size on the machine being used.

Example

```
#include <stdio.h>

int main()
{
    int    a = 10;
    float  b = 3.14;
    double c = 12345.6789;
    char   d = 'A';

    printf("int    : value = %d,   size = %zu bytes\n", a, sizeof(a));
    printf("float  : value = %f,   size = %zu bytes\n", b, sizeof(b));
    printf("double : value = %lf,  size = %zu bytes\n", c, sizeof(c));
    printf("char   : value = %c,   size = %zu bytes\n", d, sizeof(d));
    return 0;
}
```

Output

```
int    : value = 10,   size = 4 bytes
float  : value = 3.140000,   size = 4 bytes
double : value = 12345.678900,  size = 8 bytes
char   : value = A,   size = 1 bytes
```

8. Declaration and Initialization of Variables

A variable is a name given to a memory location whose value can change during execution. Every variable must be declared before it is used.

Declaration

```
data_type variable_name;
data_type variable1, variable2, variable3;
```

Declaration reserves memory for the variable but does not place any value in it. A

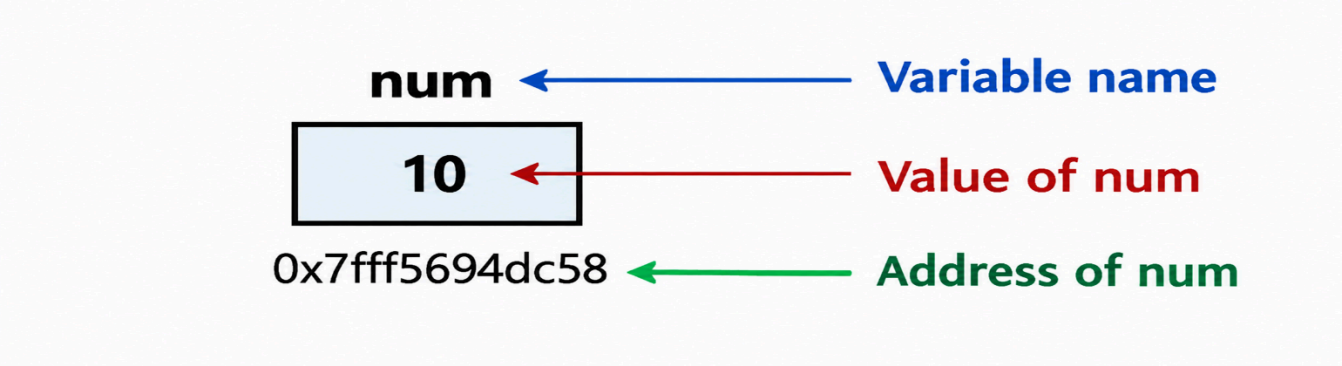
declared but uninitialised variable holds a garbage value.

Initialization

Initialization means giving the variable a value at the time of declaration.

```
data_type variable_name = value;
```

Statement	Meaning
int a;	declaration only, value is garbage
`int a = 10;`	declaration with initialization
int a, b, c;	three variables declared together
`int a = 5, b = 10;`	two variables declared and initialized
`a = 25;`	assignment to an already declared variable



Rules for naming variables

- The name must begin with a letter or an underscore.
- Blank spaces and special characters are not allowed.
- Keywords cannot be used as variable names.
- The name should suggest the purpose of the variable, such as total or avg.

Example

```
#include <stdio.h>

int main()
{
    int a, b;           /* declaration only          */
    int sum = 0;        /* declaration + initialization */
    float avg = 0.0;

    a = 25;             /* assignment after declaration */
    b = 15;
    sum = a + b;
    avg = sum / 2.0;

    printf("a = %d, b = %d\n", a, b);
    printf("sum = %d\n", sum);
    printf("avg = %.2f\n", avg);
    return 0;
```

```
}
```

Output

```
a = 25, b = 15
sum = 40
avg = 20.00
```

9. Symbolic Constants

A symbolic constant is a name given to a fixed value. The name is used throughout the program instead of the value itself. If the value has to change later, it is changed in one place only.

Using #define

```
#define name value
```

- It is a preprocessor directive, so it is written before main().
- It does not end with a semicolon.
- There is no equal sign between the name and the value.
- The preprocessor replaces every occurrence of the name with the value before compilation. No memory is reserved.
- Symbolic constant names are written in capital letters by convention.

Using const

```
const data_type name = value;
```

The const qualifier creates a read-only variable. Memory is reserved for it, it has a data type, and the compiler reports an error if the program tries to change its value.

Point	#define	const
Handled by	preprocessor	compiler
Memory used	none	memory is reserved
Data type	no data type	has a data type
Ends with ;	no	yes
Scope	from the definition to the end of the file	follows normal block scope

Example

```
#include <stdio.h>
#define MAX_MARKS 100          /* symbolic constant  */
#define NEWLINE   '\n'

int main()
{
    const float PASS_PERCENT = 33.0;   /* const qualifier */
    int obtained = 72;
    float percent;

    percent = (obtained * 100.0) / MAX_MARKS;
```

```
    printf("Marks obtained : %d out of %d%c", obtained, MAX_MARKS, NEWLINE);
    printf("Percentage     : %.2f%c", percent, NEWLINE);
    printf("Pass mark      : %.2f", PASS_PERCENT);
    return 0;
}
```

Output

```
Marks obtained : 72 out of 100
Percentage     : 72.00
Pass mark      : 33.00
```

Using enum

An enumeration defines a list of named integer constants. By default the first name is 0 and each following name is one more than the previous one. A starting value can be given, and the rest continue from it.

```
enum tag_name { NAME1, NAME2, NAME3 };
```

Example

```
#include <stdio.h>

enum weekday { MON = 1, TUE, WED, THU, FRI };

int main()
{
    enum weekday today;

    today = WED;

    printf("MON = %d\n", MON);
    printf("WED = %d\n", WED);
    printf("FRI = %d\n", FRI);
    printf("today = %d\n", today);
    return 0;
}
```

Output

```
MON = 1
WED = 3
FRI = 5
today = 3
```

10. Formatted Input and Output Functions

Formatted functions allow the data to be read or displayed in a chosen format. They use format specifiers to decide how a value is treated. printf() and scanf() are the two formatted functions, and both are declared in stdio.h.

printf()

```
printf("control string", variable1, variable2, ...);
```

The control string contains ordinary text, format specifiers and escape sequences. printf() returns the number of characters printed.

scanf()

```
scanf("control string", &variable1, &variable2, ...);
```

scanf() reads values from the keyboard and stores them in the given variables. The & symbol is the address operator, and it tells scanf() where in memory the value must be stored. It is not used with strings read into a character array.

Common format specifiers

Specifier	Used for	Specifier	Used for
%d	signed integer	%u	unsigned integer
%f	float	%lf	double
%c	single character	%s	string
%o	octal value	%x	hexadecimal value
%e	exponential form	%%	prints a percent sign

Common escape sequences

Escape sequence	Meaning	Escape sequence	Meaning
\n	new line	\t	horizontal tab
\b	backspace	\r	carriage return
\\	backslash	\"	double quote
\'	single quote	\0	null character

Example

```
#include <stdio.h>

int main()
{
    int   roll;
    char  name[20];
    float marks;

    printf("Enter roll number : ");
    scanf("%d", &roll);

    printf("Enter name        : ");
    scanf("%s", name);

    printf("Enter marks       : ");
    scanf("%f", &marks);

    printf("\nRoll  : %d\n", roll);
    printf("Name  : %s\n", name);
    printf("Marks : %.2f\n", marks);
    return 0;
}
```

Output

```
Enter roll number : 101
Enter name        : Amit
Enter marks       : 78.5

Roll  : 101
Name  : Amit
Marks : 78.50
```

Field width and precision

- %5d prints the integer in five columns, right aligned.
- %-5d prints it in five columns, left aligned.
- %.2f prints a float with two digits after the decimal point.
- %8.2f prints it in eight columns with two decimal places.

Example — formatting and escape sequences

```c
#include <stdio.h>

int main()
{
    int   roll  = 7;
    float marks = 89.5;

    printf("Name\tCourse\n");              /* \t gives a tab       */
    printf("Amit\tBCA\n\n");               /* \n gives a new line  */

    printf("Roll in 5 columns  : |%5d|\n", roll);
    printf("Roll left aligned  : |%-5d|\n", roll);
    printf("Marks, 2 decimals  : %.2f\n", marks);
    printf("Marks in 8 columns : |%8.2f|\n", marks);
    printf("A percent sign     : %d%%\n", 75);
    printf("A quote inside     : \"C language\"\n");
    return 0;
}
```

Output

```
NameCourse
AmitBCA

Roll in 5 columns  : |    7|
Roll left aligned  : |7    |
Marks, 2 decimals  : 89.50
Marks in 8 columns : |   89.50|
A percent sign     : 75%
A quote inside     : "C language"
```

11. Unformatted Input and Output Functions

Unformatted functions work with characters and strings only. They do not use format specifiers, so the data cannot be formatted, but they are faster and simpler to use.

Function	Purpose	Example

`getchar()`	reads a single character from the keyboard	ch = getchar();
`putchar()`	displays a single character	putchar(ch);
`gets()`	reads a string including blank spaces	gets(name);
`puts()`	displays a string and moves to a new line	puts(name);

gets() was removed from the C11 standard because it cannot check the size of the array, so fgets() is used in its place.

Example

```
#include <stdio.h>

int main()
{
    char ch;
    char city[30];

    printf("Enter a character : ");
    ch = getchar();              /* reads one character  */

    printf("You typed : ");
    putchar(ch);                 /* writes one character */
    putchar('\n');

    getchar();                   /* consume the newline  */

    printf("Enter a city : ");
    fgets(city, sizeof(city), stdin);   /* reads a line   */

    printf("City is : ");
    puts(city);                  /* writes a string      */
    return 0;
}
```

Output

```
Enter a character : K
You typed : K
Enter a city : Jodhpur
City is : Jodhpur
```

Difference between formatted and unformatted functions

Point	Formatted	Unformatted
Format specifiers	used	not used
Data types	works with all data types	works with characters and strings only
Formatting	output can be formatted	output cannot be formatted
Functions	printf(), scanf()	getchar(), putchar(), gets(), puts()

12. C Operators and Expressions

An operator is a symbol that performs an operation on one or more values. The values it works on are called operands. An expression is a combination of operands and operators that produces a value, for example a + b * 2.

Types of operators

Type	Operators
Arithmetic	+ - * / %
Relational	< > <= >= == !=
Logical	&& || !
Assignment	= += -= *= /= %=
Increment / Decrement	++ --
Conditional (ternary)	?:
Bitwise	& | ^ ~ << >>
Special	sizeof , & * . ->

Arithmetic operators

The modulus operator % gives the remainder and works only with integers. When both operands of / are integers, the result is an integer and the fractional part is discarded.

Example

```
#include <stdio.h>

int main()
{
    int a = 17, b = 5;

    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);      /* integer division */
    printf("a %% b = %d\n", a % b);     /* remainder        */
    return 0;
}
```

Output

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3
a % b = 2
```

Increment and decrement operators

++ increases a value by 1 and −− decreases it by 1. In the prefix form (++a) the value

changes first and is then used. In the postfix form (a++) the value is used first and changes afterwards.

Example

```
#include <stdio.h>

int main()
{
    int a = 5, b = 5;
    int p, q;

    p = a++;        /* post-increment : assign first, then increase */
    q = ++b;        /* pre-increment  : increase first, then assign */

    printf("a = %d, p = %d\n", a, p);
    printf("b = %d, q = %d\n", b, q);
    return 0;
}
```

Output

```
a = 6, p = 5
b = 6, q = 6
```

Relational and logical operators

Relational operators compare two values and produce 1 for true and 0 for false. Logical operators combine conditions: && is true when both sides are true, || is true when at least one side is true, and ! reverses the result.

Example

```
#include <stdio.h>

int main()
{
    int a = 10, b = 20;

    printf("a >  b : %d\n", a >  b);
    printf("a <  b : %d\n", a <  b);
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);

    printf("(a < b) && (b < 30) : %d\n", (a < b) && (b < 30));
    printf("(a > b) || (b > 15) : %d\n", (a > b) || (b > 15));
    printf("!(a == b)           : %d\n", !(a == b));
    return 0;
}
```

Output

```
a >  b : 0
a <  b : 1
a == b : 0
a != b : 1
(a < b) && (b < 30) : 1
(a > b) || (b > 15) : 1
!(a == b)           : 1
```

Assignment operators

Operator	Example	Same as
=	a = 10	a = 10
+=	a += 5	a = a + 5
−=	a −= 5	a = a − 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5

Example

```
#include <stdio.h>

int main()
{
    int a = 20;

    printf("a      = %d\n", a);

    a += 5;   printf("a += 5 = %d\n", a);
    a -= 3;   printf("a -= 3 = %d\n", a);
    a *= 2;   printf("a *= 2 = %d\n", a);
    a /= 4;   printf("a /= 4 = %d\n", a);
    a %= 5;   printf("a %%= 5 = %d\n", a);
    return 0;
}
```

Output

```
a      = 20
a += 5 = 25
a -= 3 = 22
a *= 2 = 44
a /= 4 = 11
a %= 5 = 1
```

Conditional operator

```
expression1 ? expression2 : expression3
```

If expression1 is true, the value of expression2 is taken, otherwise the value of expression3 is taken. It is the only operator in C that takes three operands.

Example

```
#include <stdio.h>

int main()
{
    int marks = 45;
    char result;

    result = (marks >= 33) ? 'P' : 'F';

    printf("Marks  : %d\n", marks);
```

```
    printf("Result : %c\n", result);
    return 0;
}
```

Output

```
Marks  : 45
Result : P
```

Bitwise operators

Bitwise operators work on the individual bits of an integer. For a = 12 (1100) and b = 10 (1010), & gives 1 only where both bits are 1, | gives 1 where at least one bit is 1, and ^ gives 1 where the bits differ. << shifts bits to the left and >> shifts them to the right.

Example

```
#include <stdio.h>

int main()
{
    int a = 12, b = 10;     /* 12 = 1100 , 10 = 1010 */

    printf("a & b  = %d\n", a & b);
    printf("a | b  = %d\n", a | b);
    printf("a ^ b  = %d\n", a ^ b);
    printf("a << 1 = %d\n", a << 1);
    printf("a >> 1 = %d\n", a >> 1);
    return 0;
}
```

Output

```
a & b  = 8
a | b  = 14
a ^ b  = 6
a << 1 = 24
a >> 1 = 6
```

Special operators

- sizeof — returns the size of a variable or data type in bytes.
- comma (,) — joins related expressions, evaluated from left to right.
- address (&) — gives the memory address of a variable.
- indirection (*) — gives the value stored at an address.
- dot (.) and arrow (−>) — used to access members of a structure.

Example

```
#include <stdio.h>

int main()
{
    int   a = 50;
    float f = 2.5;
    int  *p;
```

```
    p = &a;                 /* & gives the address of a       */

    printf("sizeof(int)   = %zu bytes\n", sizeof(int));
    printf("sizeof(a)     = %zu bytes\n", sizeof(a));
    printf("sizeof(f)     = %zu bytes\n", sizeof(f));
    printf("value of a    = %d\n", a);
    printf("value at p    = %d\n", *p);   /* * gives value at address */
    return 0;
}
```

Output

```
sizeof(int)   = 4 bytes
sizeof(a)     = 4 bytes
sizeof(f)     = 4 bytes
value of a    = 50
value at p    = 50
```

13. Evaluation of Arithmetic Expressions

When an expression contains more than one operator, C decides the order of evaluation using two rules: precedence and associativity.

- Precedence decides which operator is applied first when different operators appear together.
- Associativity decides the direction when two operators of the same precedence appear together.

Precedence and associativity

Priority	Operators	Associativity
1	( ) [ ] . ->	left to right
2	! ~ ++ -- + - (type) sizeof	right to left
3	* / %	left to right
4	+ -	left to right
5	<< >>	left to right
6	< <= > >=	left to right
7	== !=	left to right
8	& ^ |	left to right
9	&& then ||	left to right
10	? :	right to left
11	= += -= *= /= %=	right to left
12	,	left to right

Step by step evaluation

Consider the expression 2 + 3 * 4 − 6 / 2

Step	Operation applied	Expression becomes

1	3 * 4 (highest precedence, leftmost)	2 + 12 − 6 / 2
2	6 / 2 (same precedence as *, next to the right)	2 + 12 − 3
3	2 + 12 (left to right)	14 − 3
4	14 − 3	11

Rules to remember

- Brackets are evaluated first. Inner brackets are evaluated before outer ones.
- Within the same precedence level, evaluation goes from left to right for most operators.
- Integer division discards the fractional part: 10 / 4 gives 2, not 2.5.
- To get a real result, at least one operand must be a real number: 10.0 / 4 gives 2.50.
- The modulus operator cannot be used with float or double values.

Example

```
#include <stdio.h>

int main()
{
    int result;

    result = 2 + 3 * 4 - 6 / 2;
    printf("2 + 3 * 4 - 6 / 2 = %d\n", result);

    result = (2 + 3) * (4 - 6) / 2;
    printf("(2 + 3) * (4 - 6) / 2 = %d\n", result);

    printf("10 / 4     = %d\n", 10 / 4);      /* integer division */
    printf("10 %% 4     = %d\n", 10 % 4);
    printf("10.0 / 4   = %.2f\n", 10.0 / 4);  /* real division    */
    return 0;
}
```

Output

```
2 + 3 * 4 - 6 / 2 = 11
(2 + 3) * (4 - 6) / 2 = -5
10 / 4     = 2
10 % 4     = 2
10.0 / 4   = 2.50
```

14. Type Conversion

Type conversion means changing a value of one data type into another. C performs some conversions automatically, and the programmer can force others.

Implicit type conversion

The compiler performs this conversion on its own when an expression mixes different data types. The lower data type is promoted to the higher one, so that no

information is lost. This is also called type promotion.

```
char  →  short  →  int  →  unsigned int  →  long
      →  unsigned long  →  float  →  double  →  long double
```

- The result of the expression takes the higher data type.
- int + float gives a float result.
- A char used in an arithmetic expression is converted to its ASCII value.
- Information can be lost when a higher type is assigned to a lower type, for example float to int.

Explicit type conversion (type casting)

The programmer converts a value to a required type using the cast operator. The conversion is temporary and applies only to that expression; the variable itself is not changed.

```
(data_type) expression
```

Expression	Result	Reason
`7 / 2`	3	both operands are int, so integer division
`(float) 7 / 2`	3.50	7 is cast to float, so real division
`(int) 9.99`	9	the fractional part is discarded, not rounded
`(char) 66`	B	66 is the ASCII code of B

Example

```
#include <stdio.h>

int main()
{
    int   i = 7, j = 2;
    float f;
    char  ch = 'A';

    f = i / j;                  /* implicit : int / int = int   */
    printf("i / j           = %.2f\n", f);

    f = (float) i / j;          /* explicit : cast to float     */
    printf("(float) i / j   = %.2f\n", f);

    printf("ch + 1          = %d\n", ch + 1);   /* char promoted to int */
    printf("(char)(ch + 1)  = %c\n", (char)(ch + 1));
    printf("(int) 9.99      = %d\n", (int) 9.99);
    return 0;
}
```

Output

```
i / j           = 3.00
(float) i / j   = 3.50
ch + 1          = 66
(char)(ch + 1)  = B
(int) 9.99      = 9
```

Difference between implicit and explicit conversion

Point	Implicit	Explicit
Performed by	the compiler	the programmer
Also called	type promotion	type casting
Direction	lower type to higher type	any type to any type
Loss of data	normally no loss	loss is possible
Syntax	nothing is written	(type) is written before the value

15. Practice Programs

The following programs use only the ideas covered in this unit: variables, operators, expressions, type conversion and formatted input and output. In the sample runs, the values shown after each prompt are typed by the user.

Program 1: Swap two numbers using a third variable

```
/* Swap two numbers using a third variable */
#include <stdio.h>

int main()
{
    int a, b, temp;

    printf("Enter first number  : ");
    scanf("%d", &a);
    printf("Enter second number : ");
    scanf("%d", &b);

    printf("\nBefore swapping : a = %d, b = %d\n", a, b);

    temp = a;        /* store a in temp  */
    a    = b;        /* copy b into a    */
    b    = temp;     /* copy temp into b */

    printf("After  swapping : a = %d, b = %d\n", a, b);
    return 0;
}
```

Output

```
Enter first number  : 10
Enter second number : 20

Before swapping : a = 10, b = 20
After  swapping : a = 20, b = 10
```

A third variable holds the first value while the two variables exchange their contents. Without it, the first value would be lost as soon as b is copied into a.

Program 2: Swap two numbers without a third variable

```
/* Swap two numbers without using a third variable */
#include <stdio.h>
```

```
int main()
{
    int a, b;

    printf("Enter first number  : ");
    scanf("%d", &a);
    printf("Enter second number : ");
    scanf("%d", &b);

    printf("\nBefore swapping : a = %d, b = %d\n", a, b);

    a = a + b;       /* a now holds the sum        */
    b = a - b;       /* b becomes the original a   */
    a = a - b;       /* a becomes the original b   */

    printf("After  swapping : a = %d, b = %d\n", a, b);
    return 0;
}
```

Output

```
Enter first number  : 10
Enter second number : 20

Before swapping : a = 10, b = 20
After  swapping : a = 20, b = 10
```

- a = a + b stores the sum in a.
- b = a − b removes the original b from the sum, leaving the original a in b.
- a = a − b removes the new b from the sum, leaving the original b in a.
- The same swap can be done with the bitwise XOR operator: a = a ^ b; b = a ^ b; a = a ^ b;
- The addition method can overflow if the two numbers are very large, so the method using a third variable is safer in practice.

Program 3: Area and circumference of a circle

```
/* Area and circumference of a circle */
#include <stdio.h>
#define PI 3.14159

int main()
{
    float radius, area, circumference;

    printf("Enter the radius : ");
    scanf("%f", &radius);

    area          = PI * radius * radius;
    circumference = 2 * PI * radius;

    printf("\nRadius        = %.2f\n", radius);
    printf("Area          = %.2f\n", area);
    printf("Circumference = %.2f\n", circumference);
    return 0;
}
```

Output

```
Enter the radius : 7

Radius        = 7.00
Area          = 153.94
Circumference = 43.98
```

Area = πr^2 and Circumference = $2\pi r$. PI is kept as a symbolic constant so the value appears in one place only.

Program 4: Area and perimeter of a rectangle

```
/* Area and perimeter of a rectangle */
#include <stdio.h>

int main()
{
    float length, breadth, area, perimeter;

    printf("Enter the length  : ");
    scanf("%f", &length);
    printf("Enter the breadth : ");
    scanf("%f", &breadth);

    area      = length * breadth;
    perimeter = 2 * (length + breadth);

    printf("\nArea      = %.2f\n", area);
    printf("Perimeter = %.2f\n", perimeter);
    return 0;
}
```

Output

```
Enter the length  : 12
Enter the breadth : 5

Area      = 60.00
Perimeter = 34.00
```

Area = length × breadth and Perimeter = 2 × (length + breadth).

Program 5: Area and perimeter of a square

```
/* Area and perimeter of a square */
#include <stdio.h>

int main()
{
    float side, area, perimeter;

    printf("Enter the side : ");
    scanf("%f", &side);

    area      = side * side;
    perimeter = 4 * side;

    printf("\nArea      = %.2f\n", area);
    printf("Perimeter = %.2f\n", perimeter);
    return 0;
}
```

Output

```
Enter the side : 6

Area      = 36.00
Perimeter = 24.00
```

Area = side^2 and Perimeter = 4 × side.

Program 6: Convert Fahrenheit to Celsius

```
/* Convert Fahrenheit to Celsius */
#include <stdio.h>

int main()
{
    float fahrenheit, celsius;

    printf("Enter temperature in Fahrenheit : ");
    scanf("%f", &fahrenheit);

    celsius = (fahrenheit - 32) * 5 / 9;

    printf("\n%.2f F = %.2f C\n", fahrenheit, celsius);
    return 0;
}
```

Output

```
Enter temperature in Fahrenheit : 98.6

98.60 F = 37.00 C
```

The formula is C = (F − 32) × 5 / 9. Because fahrenheit is a float, the whole expression is evaluated as a real number and the fraction is not lost.

Program 7: Convert Celsius to Fahrenheit

```
/* Convert Celsius to Fahrenheit */
#include <stdio.h>

int main()
{
    float celsius, fahrenheit;

    printf("Enter temperature in Celsius : ");
    scanf("%f", &celsius);

    fahrenheit = (celsius * 9 / 5) + 32;

    printf("\n%.2f C = %.2f F\n", celsius, fahrenheit);
    return 0;
}
```

Output

```
Enter temperature in Celsius : 37

37.00 C = 98.60 F
```

The formula is F = (C × 9 / 5) + 32. If the variables were declared as int, 9 / 5 would give 1 instead of 1.8 and the answer would be wrong.